BẢO TOÀN LUỒNG VÀ TRẠNG THÁI TRONG BÀI TOÁN SOICT 2026

(Integrated Passenger and Parcel Transportation)

Báo cáo Kiến trúc Thuật toán & Mã nguồn Python

Ngày 16 tháng 9 năm 2026

1 Tổng quan về "Bảo toàn luồng" (Flow Conservation)

Trong các bài toán định tuyến phương tiện phức hợp như **PDVRPTW** (Pickup and Delivery Vehicle Routing Problem with Time Windows), khái niệm **"Bảo toàn luồng"** (Flow Conservation) và **Bảo toàn trạng thái** thể hiện sự liên tục không đứt gãy về *Không gian*, *Thời gian*, *Tải trọng* và *Logic dịch vụ* khi xe taxi di chuyển qua lộ trình:

$$R_k = (v_0^k, v_1^k, \ldots, v_{m_k}^k)$$

Trong mã nguồn Python của dự án, logic này được phân bổ chặt chẽ giữa các file:

- `models.py`: Khai báo các biến trạng thái tích lũy ($current_load, current_time, stops$).
- `validator.py`: Viết hàm kiểm tra toàn bộ các điều kiện bảo toàn luồng xem lộ trình có hợp lệ hay không.
- `greedy_solver.py`: Đảm bảo khi chèn (Insert) một nút mới vào tuyến thì quy luật bảo toàn luồng không bị phá vỡ.
- `utils.py`: Hàm phụ trợ tính toán sự thay đổi trạng thái ($\Delta time, \Delta load$) giữa hai điểm liền kề.

—

2 Chi tiết 4 Điều kiện Bảo toàn Luồng & Vị trí trong Code Python

2.1 1. Bảo toàn Luồng Không gian (Spatial Flow Conservation)

- **Bản chất toán học**: Nếu taxi k di chuyển vào một nút dịch vụ u, nó bắt buộc phải rời nút u để đi tiếp đến nút v. Luồng bắt đầu từ điểm xuất phát O_k (Depot) và kết thúc tại nút dịch vụ cuối cùng của chuyến xe.

- **Phương trình toán học**:

$$\sum_j x_{u,j}^k - \sum_i x_{i,u}^k = 0 \quad \forall u \in \text{nút trung gian} \tag{1}$$

- **Vị trí trong Code Python**:
  - `models.py`: Lưu danh sách tuyến đường bên trong thuộc tính `Route.stops: List[Node]`. Đây là một chuỗi điểm nối tiếp $O_k \to v_1^k \to v_2^k \dots$
  - `validator.py`: Sử dụng vòng lặp duyệt liên tiếp để đảm bảo tính liên tục trong không gian:

```
for i in range(len(route.stops) - 1):
    u = route.stops[i]
    v = route.stops[i + 1]
    # Cung di chuyển (u, v) phải tồn tại trong ma trận khoảng cách
```

2.2 2. Bảo toàn Luồng Thời gian (Temporal Flow Conservation)

- **Bản chất toán học**: Thời gian là dòng chảy một chiều gia tăng. Thời gian đến điểm sau arr_v bằng thời gian rời điểm trước dep_u cộng thời gian di chuyển $t(u, v)$. Xe không được đến muộn hơn thời gian đóng cửa (L_v). Nếu đến sớm hơn thời gian mở cửa (E_v), xe phải chờ.

- **Phương trình toán học**:

$$arr_v = dep_u + t(u, v) \tag{2}$$

$$srv_v = \max(arr_v, E_v) \quad \text{với điều kiện } arr_v \leq L_v \tag{3}$$

$$dep_v = srv_v + S_v \tag{4}$$

- **Vị trí trong Code Python**:
  - `models.py`: Thuộc tính `Route.current_time` lưu mốc thời gian lũy kế tại điểm hiện tại.
  - `utils.py`: Viết hàm tính toán mốc thời gian đến và rời đi `calculate_next_time(dep_u, u, v, time_matrix)`.
  - `validator.py`: Cập nhật tích lũy `current_time` qua từng điểm dịch vụ và dừng chương trình (trả về `False`) ngay khi `arrival_time > late_window`.

2.3 3. Bảo toàn Luồng Tải trọng (Capacity Flow Conservation)

- **Bản chất toán học**: Tải trọng hàng hóa trên xe là đại lượng tích lũy động. Khi qua điểm lấy hàng bưu kiện P_j^h, luồng hàng cộng thêm w_j. Khi qua điểm giao hàng D_j^h, luồng hàng giảm đi w_j. Tại mọi thời điểm, tải trọng luôn bị chặn bởi sức chứa taxi Q_k.

- **Phương trình toán học**:

$$load_v = \begin{cases} load_u + w_j & \text{nếu } v = P_j^h \text{ (lấy hàng)} \\ load_u - w_j & \text{nếu } v = D_j^h \text{ (giao hàng)} \end{cases} \tag{5}$$

$$0 \leq load_v \leq Q_k \quad \forall v \in R_k \tag{6}$$

- **Vị trí trong Code Python**:
  - `models.py`: Thuộc tính `Route.current_load` ghi nhận lượng hàng hiện có trên xe.
  - `validator.py`: Cập nhật `current_load += req.weight` (khi gặp `PARCEL_PICKUP`) và `current_load -= req.weight` (khi gặp `PARCEL_DROPOFF`). Kiểm tra nếu `current_load > taxi.capacity` thì trả về lỗi vi phạm.

2.4 4. Bảo toàn Luồng Trực tiếp Hành khách (Direct Passenger Flow)

- **Bản chất toán học**: Luồng chở khách là luồng kín độc lập và không thể ngắt quãng. Ngay sau khi đón hành khách i tại P_i^n, xe phải đi trực tiếp tới điểm trả D_i^n mà không được dừng ở bất kỳ điểm nào khác ở giữa.

- **Phương trình toán học**:

$$\text{Nếu } v_r^k = P_i^n \implies v_{r+1}^k = D_i^n \quad \forall i \in S_N \tag{7}$$

- **Vị trí trong Code Python**:
  - `validator.py`: Kiểm tra vị trí kế tiếp trong danh sách nút `stops`:

```
if node.type == 'PASSENGER_PICKUP':
    if i + 1 >= len(stops) or stops[i + 1] != req.dropoff_node:
        return False # Vi phạm luồng đi thẳng chở khách!
```

  - `greedy_solver.py`: Khi thiết kế hàm chèn hành khách vào tuyến xe, luôn chèn cặp song song $[P_i^n, D_i^n]$ kề nhau liên tiếp vào chuỗi lộ trình.

—

3 Bảng Tóm Tắt Ánh Xạ Logic Luồng Vào Mã Nguồn Python

Bảng 1: Bảng ánh xạ các quy tắc Bảo toàn luồng vào các File Python

Loại Bảo Toàn Luồng	**File Python Đảm Nhận**	**Thành Phần / Hàm Chính**
Cấu trúc Trạng thái	`models.py`	Class `Route` $(current_load, current_time)$
Luồng Không gian	`validator.py`	Duyệt lặp qua chỉ số `stops[i]` $\rightarrow$ `stops[i+1]`
Luồng Thời gian	`utils.py` / `validator.py`	Hàm tính `arrival_time` & so sánh với L_v
Luồng Tải trọng	`validator.py`	Cộng/trừ `weight` và so sánh với `capacity`
Luồng Khách Trực tiếp	`greedy_solver.py` & `validator.py`	Bắt buộc `stops[i+1]` là `Passenger_Dropoff`